

1 Hash Indices and Hashing Techniques

Overview

Hashing is a fundamental technique used to organize and access data efficiently. In database systems, hashing is employed to build **main-memory indices** and sometimes as a method of **file organization**.

Hash-based indices are primarily used for:

- Fast **equality search** (e.g., `WHERE id = 1001`)
- Temporary in-memory structures (e.g., during joins)
- Permanent indices in main-memory databases

1.1 Basic Concepts of Hashing

- A **bucket** is a storage unit that can hold one or more records.
- For in-memory indices, a bucket is typically a **linked list of records or pointers**.
- For disk-based indices, a bucket corresponds to one or more **disk blocks**.

Formal Definition

Let:

K = set of all search-key values, B = set of bucket addresses

A **hash function** $h : K \rightarrow B$ maps each key to a bucket.

The hash index maintains an array of bucket pointers. For a given key K_i , its record is placed into the bucket at position $h(K_i)$.

1.2 Insertion and Lookup in Hash Indices

Insertion Algorithm

1. Compute the hash value $h(K_i)$.
2. Locate the corresponding bucket.
3. Insert the record (or pointer) into the linked list at that bucket.

Lookup Algorithm

1. Compute $h(K_i)$.
2. Search the records in that bucket.
3. Verify each record's key to ensure an exact match.

Efficiency Note

Hash indices provide excellent performance for **equality queries**, but do **not support range queries** (e.g., $10 < K < 20$).

1.3 Overflow Chaining (Closed Addressing)

Concept: Overflow Chaining

When multiple keys hash to the same bucket, a **linked list of overflow records** is maintained. This technique is called **closed addressing** or **overflow chaining**.

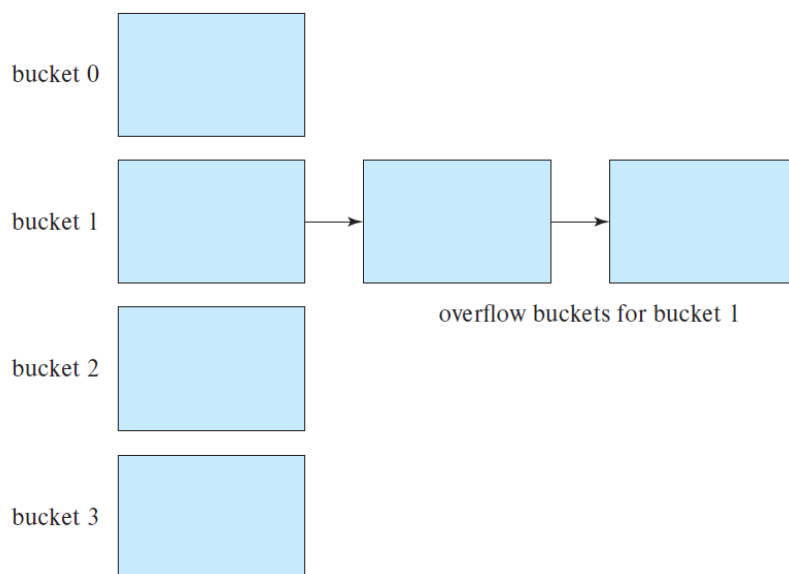


Figure 1: Overflow chaining in a disk-based hash structure.

Insertion with Overflow:

- If a bucket is full, allocate an **overflow bucket**.
- Link it to the existing chain.
- Continue chaining as necessary.

Lookup:

- Search all records in bucket $h(K_i)$.
- Follow all overflow bucket links sequentially.

Deletion: Simply remove the record from the appropriate linked list node.

1.4 Bucket Overflow and Skew

Bucket Overflow

Occurs when a bucket has insufficient space to store all records assigned to it.

Causes:

- Too few buckets allocated initially.
- Nonuniform distribution (bad hash function).
- Data skew — many identical search keys.

To minimize overflow:

$$\text{Number of buckets} = \frac{n_r}{f_r} \times (1 + d)$$

where:

- n_r = number of records
- f_r = records per bucket
- d = fudge factor (typically 0.2)

This leaves about 20% of space empty to reduce overflow probability.

1.5 Static Hashing

Definition

If the number of buckets is **fixed at index creation time**, the scheme is called **Static Hashing**.

Problem: As more records are added, buckets overflow and long overflow chains form, reducing lookup efficiency.

Possible Solution: Rebuild the index with more buckets — but this is expensive for large databases.

1.6 Dynamic Hashing Techniques

When the number of records changes significantly over time, static hashing becomes inefficient. To overcome this, databases use **dynamic hashing**, where the number of buckets can grow or shrink incrementally.

1.6.1 Periodic Rehashing

Key Idea

The hash table is rebuilt periodically when the load factor exceeds a threshold, redistributing all keys into a larger table.

Mechanism:

1. Monitor the **load factor** $\alpha = \frac{\text{Number of records}}{\text{Number of buckets}}$.
2. When α exceeds a threshold (e.g., 0.7), allocate a new table with more buckets.
3. Rehash all existing records into the new table using the updated hash function.

Advantages:

- Simple to implement.
- Maintains good performance by controlling the load factor.

Disadvantages:

- Can be expensive since all records are rehashed at once.
- May cause temporary performance spikes during rehashing.

1.6.2 Linear Hashing

Key Idea

Buckets are split **gradually** as the file grows, without rebuilding the entire hash table.

Mechanism:

1. Maintain a hash function $h_i(K) = K \bmod 2^i N$.
2. When overflow occurs, split the next bucket in sequence.
3. Gradually all buckets move from level i to $i + 1$.

Advantages:

- No global rehashing needed.
- Smooth growth proportional to data size.

1.6.3 Extendible Hashing

Key Idea

Uses a **directory** of pointers to buckets, allowing flexible expansion without full reorganization.

Mechanism:

- Each key is hashed to a binary value.
- Only a prefix of the hash (first i bits) is used to locate a bucket.
- When a bucket overflows:
 - Split the bucket.
 - If needed, double the directory size.

Advantages:

- Supports incremental growth.
- Lookup and insertion are efficient (single bucket access).

1.7 Comparison of Hashing Methods

Aspect	Static Hashing	Dynamic Hashing
Bucket Count	Fixed	Variable (grows/shrinks)
Overflow Handling	Overflow chains	Splitting/Directory expansion
Reorganization	Full rebuild needed	Incremental adjustment
Performance	Degrades with growth	Stable with data growth
Techniques	Overflow chaining	Linear / Extendible Hashing

Summary

- Hash indices are ideal for **equality queries**.
- Overflow chaining manages collisions effectively but can grow long.

- Static hashing is simple but rigid.
- Dynamic hashing (Linear / Extendible) allows **adaptive resizing**.